BLOG · MACHINE LEARNING

Project Title:

Detecting Data Leakage in ML

Why a model with 99% accuracy can collapse in production, and how to catch the leak before you deploy



Author: **Deepak Singh**

AI Engineer | LLM Applications, RAG & Production ML

Date and Version: September 2026 · v1.0

Important Links:

Portfolio | LinkedIn | GitHub | Resume

Detecting Data Leakage in Machine Learning

A practitioner's guide to finding the silent bug that makes models look brilliant in the notebook and fail in production

Introduction

Every experienced ML engineer has lived this moment: a model hits 99% accuracy on the first run, the team celebrates, and three weeks after deployment the business metrics collapse. The cause is rarely the algorithm. It is almost always **data leakage**.

Data leakage happens when information that would not be available at prediction time finds its way into model training or evaluation. The model does not learn the real signal. It learns a shortcut, and shortcuts disappear in production.

Leakage is dangerous for three reasons:

1. **It is silent.** No error, no warning, only suspiciously good numbers.
2. **It is common.** Studies of published ML research have found leakage across dozens of fields, affecting hundreds of papers, in domains from medicine to civil war prediction.
3. **It is expensive.** Wrong models drive wrong decisions, wasted compute, lost trust and, in healthcare or finance, real harm.

This guide covers what leakage is, its full taxonomy, how to detect each type with concrete techniques and code, how to audit pipelines, and how to build systems where leakage is hard to introduce in the first place.

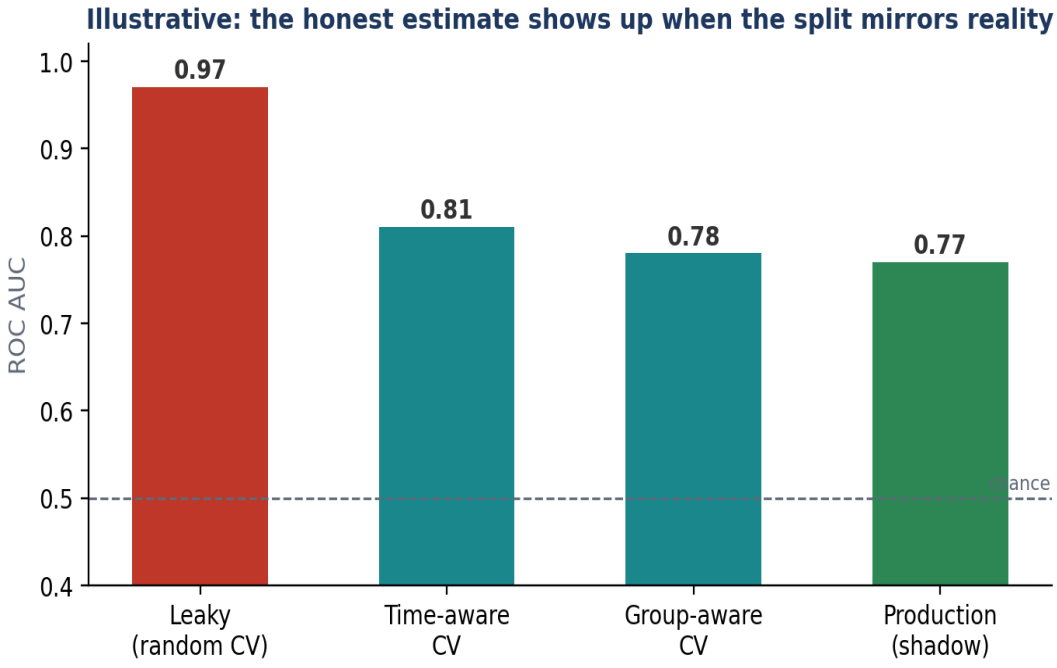


Figure 1. Illustrative: the honest estimate appears once the evaluation mirrors reality.

1. What Is Data Leakage?

Formally, leakage occurs when the training process has access to information about the target that will not exist when the model is used for real predictions, or when the evaluation set is not truly independent of the training set.

There are two broad families:

- **Target (feature) leakage:** a feature contains information derived from, or only available after, the label.
- **Train-test contamination:** information flows between the training data and the validation or test data

> **Core idea**
> Both families produce the same symptom: **an optimistic gap between offline performance and real-world performance.**

2. A Taxonomy of Leakage

Knowing the types is half of detection. You cannot find what you do not know to look for.

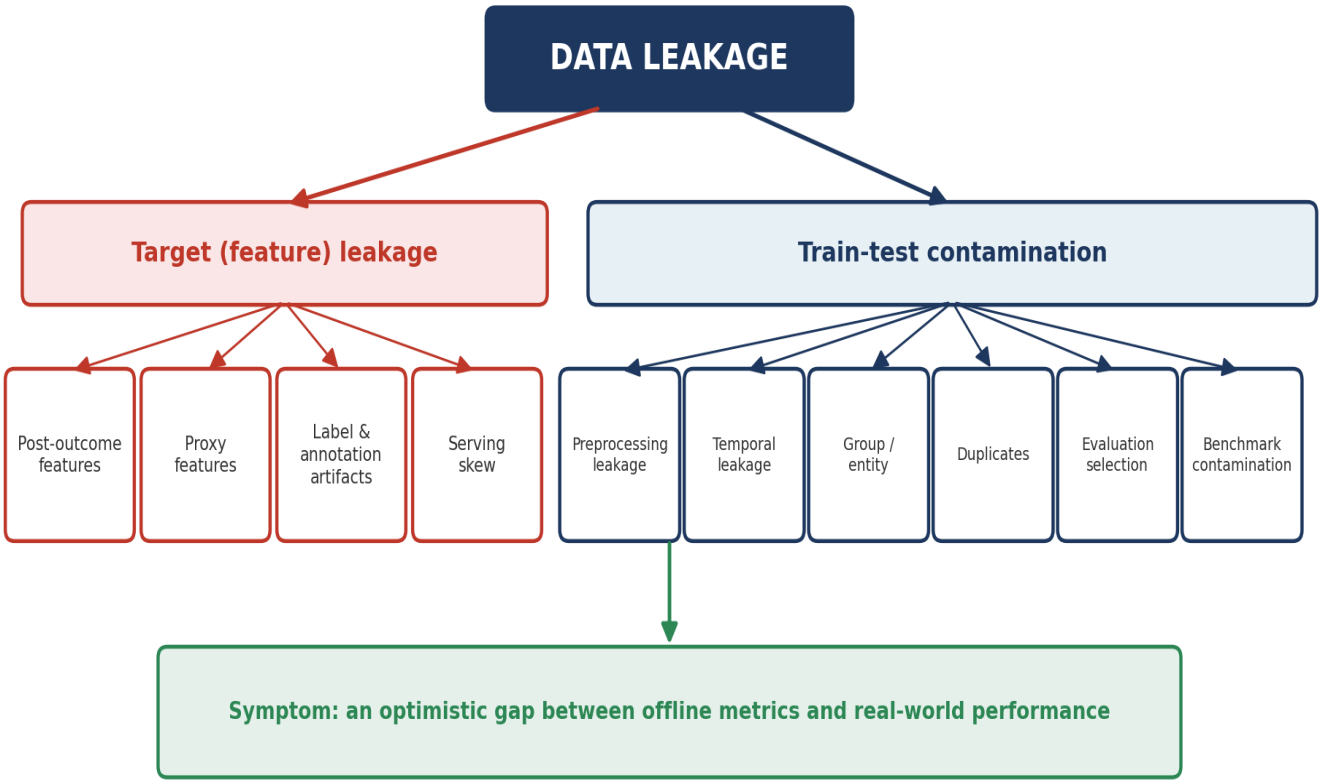


Figure 2. Taxonomy of data leakage.

2.1 Target Leakage

A feature is a proxy for, or a consequence of, the label.

- Predicting loan default using `collections_status` or `days_past_due`, which only exist after the borrower defaults.
- Predicting hospital readmission using `discharge_summary_length` when summaries are written after outcomes are known.
- Predicting customer churn using `account_closure_date`.
- Predicting fraud using `chargeback_flag`.

2.2 Preprocessing Leakage

Transformations are fit on the full dataset before splitting.

- Scaling or normalizing using global mean and standard deviation.
- Imputing missing values using statistics from the whole dataset.
- Running PCA, feature selection or target encoding before the split.
- Oversampling (SMOTE) before the split, which puts synthetic copies of test-adjacent points into training.

This is the most common form in tutorials and notebooks, and it often inflates scores subtly rather than dramatically, which makes it harder to notice.

2.3 Temporal Leakage

The model sees the future.

- Random shuffling of time-ordered data, so the model trains on Tuesday and is tested on Monday.
- Rolling features computed with windows that include future rows.
- Labels defined over a horizon that overlaps with the feature window.
- Using the latest snapshot of a slowly changing table (for example, a customer's current tier) to explain a historical event.

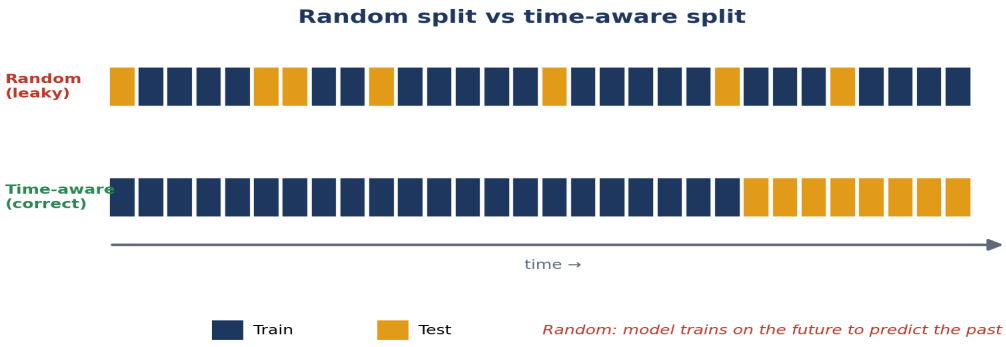


Figure 3. Random split versus time-aware split.

2.4 Group and Entity Leakage

The same entity appears in both train and test.

- Multiple images or scans from the same patient split across sets.
- Multiple sessions from the same user, or transactions from the same merchant.
- Frames from the same video.

The model memorizes the entity instead of learning a generalizable pattern. Medical imaging is a notorious victim.

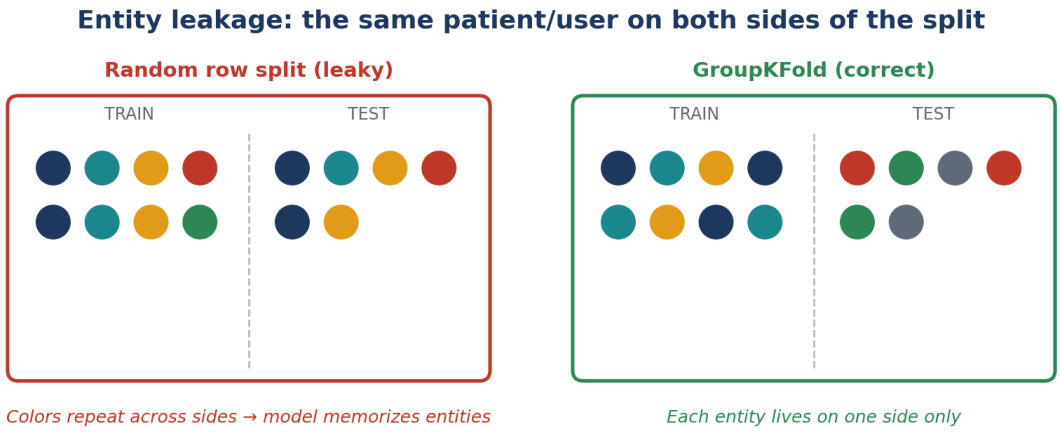


Figure 4. Entity leakage and its fix with group-aware splitting.

2.5 Duplicate and Near-Duplicate Leakage

Identical or almost identical records exist across splits: repeated rows, augmented copies, paraphrased texts, resized images, or the same document scraped twice from different URLs.

2.6 Label and Annotation Leakage

- Labels derived from a model whose inputs overlap with your features (for example, weak labels from a rules engine that uses the same columns).
- Annotation artifacts: a file naming scheme, image watermark, scanner type or hospital logo that correlates with the label.
- Data collection bias: positive and negative samples collected from different sources, so the model learns the source, not the condition.

2.7 Evaluation and Selection Leakage

- Tuning hyperparameters on the test set, or early stopping on it.
- Repeatedly peeking at test performance and iterating until it looks good (leakage through the human).
- Reporting the best of many runs or seeds on the same test data.
- Using cross-validation scores for both model selection and final reporting without nested cross-validation.

2.8 Benchmark Contamination (LLMs and Foundation Models)

Test data from public benchmarks appears in the pretraining corpus or fine-tuning mix. The model has effectively seen the answers. This has become one of the central evaluation problems for large language models.

2.9 Production and Pipeline Leakage

- Features computed differently (or with different latency) in training and serving.
- Backfilled features that include data which arrived late in reality.
- Feature stores without point-in-time correctness.

3. Warning Signs: Smell Tests Before Any Code

Before running diagnostics, look for these red flags:

Red flag	What it suggests
Near-perfect metrics (AUC above 0.98) on a hard, noisy problem	Target leakage or duplicates
Validation score far above what a baseline or domain expert would expect	Leakage of some kind
One feature dominates importance	Proxy for the label
Large gap between offline and online performance	Train-serving skew or leakage
Sharp drop when switching from random to time-based split	Temporal leakage
Sharp drop when switching to group-based split	Entity leakage
Train and validation both implausibly high with no overfitting gap	Leakage affects both sets equally
Model performs well with only metadata (IDs, filenames, timestamps)	Artifact leakage

> **Mindset**
> **Treat every great result as a bug until proven otherwise.**

4. Detection Techniques

4.1 Single-Feature Predictive Power Test

Train a tiny model on each feature alone. Any single feature with extraordinary performance deserves interrogation.

```
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_predict
from sklearn.tree import DecisionTreeClassifier

def single_feature_auc(X: pd.DataFrame, y, max_depth=3):
    results = {}
    for col in X.columns:
        clf = DecisionTreeClassifier(max_depth=max_depth, random_state=0)
        xi = X[[col]].fillna(-999)
        proba = cross_val_predict(clf, xi, y, cv=5, method='predict_proba')[:, 1]
        results[col] = roc_auc_score(y, proba)
    return pd.Series(results).sort_values(ascending=False)

print(single_feature_auc(X, y).head(10))
```

If one column alone yields AUC above 0.9 on a problem where that makes no domain sense, investigate how it is generated.

4.2 Feature Importance Audit

Use multiple lenses, since each has different blind spots:

- **Built-in importance** (gain, split count) for a quick look.
- **Permutation importance** on a held-out set, which is model-agnostic and less biased toward high-cardinality features.
- **SHAP values** to see direction, interaction and per-sample behavior.

```
from sklearn.inspection import permutation_importance

r = permutation_importance(model, X_val, y_val, n_repeats=10,
                           random_state=0, scoring='roc_auc')
imp = pd.Series(r.importances_mean, index=X_val.columns).sort_values(ascending=False)
print(imp.head(15))
```

For every top feature ask the **availability question**: *At the exact moment of prediction in production, will this value exist, and will it have this value?* If the answer is no or maybe, you have found a suspect.

4.3 Correlation and Mutual Information Scan

Screen for features with unusually high correlation or mutual information with the target.

```
from sklearn.feature_selection import mutual_info_classif

mi = mutual_info_classif(X.fillna(-999), y, random_state=0)
print(pd.Series(mi, index=X.columns).sort_values(ascending=False).head(10))
```

Leakage can be nonlinear or distributed across several features, so a clean correlation matrix proves nothing on its own.

4.4 Adversarial Validation

One of the most powerful and underused techniques. Train a classifier to distinguish **train rows from test rows**. If it cannot (AUC near 0.5), the splits look alike. If it can, the splits differ, which signals drift, a biased split, or artifacts that identify the source.

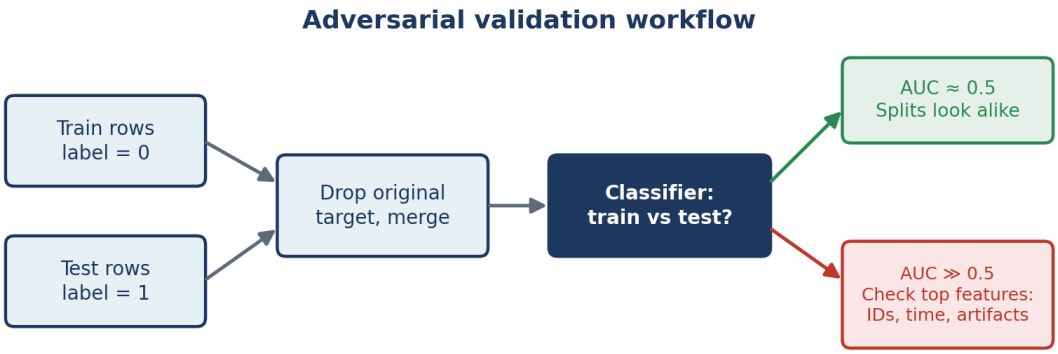


Figure 5. Adversarial validation workflow.

```
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import cross_val_score

X_all = pd.concat([X_train, X_test])
is_test = np.r_[np.zeros(len(X_train)), np.ones(len(X_test))]

clf = lgb.LGBMClassifier(n_estimators=200, learning_rate=0.05, verbose=-1)
auc = cross_val_score(clf, X_all, is_test, cv=5, scoring='roc_auc').mean()
print('Adversarial AUC:', round(auc, 3))
```

Inspect the feature importances of the adversarial model: the features that separate train from test carry split artifacts, timestamps or IDs.

4.5 Shuffled-Label (Permutation) Test

Randomly permute the labels and re-run your **entire pipeline**. A correct pipeline on shuffled labels should score at chance level. If it scores meaningfully above chance, information is leaking through something other than the true label relationship, such as duplicated rows, group structure or preprocessing.

```
rng = np.random.default_rng(0)
y_shuffled = rng.permutation(y)
score = run_full_pipeline(X, y_shuffled)   # your CV pipeline, end to end
print('Shuffled-label score:', score)      # should be about 0.5 AUC
```

A related **label-noise sanity check**: score should degrade smoothly as you corrupt labels.

4.6 Time-Aware Validation Comparison

Compare two evaluations on the same data: random K-fold and forward-chaining time split (train on past, validate on future).

```
from sklearn.model_selection import KFold, TimeSeriesSplit, cross_val_score

random_cv = cross_val_score(model, X, y,
    cv=KFold(5, shuffle=True, random_state=0), scoring='roc_auc').mean()
time_cv = cross_val_score(model, X, y,
    cv=TimeSeriesSplit(5), scoring='roc_auc').mean()
print(random_cv, time_cv)
```

A big drop from random to time-based CV points to temporal leakage or non-stationarity. Both deserve attention, but only the first is a bug in your evaluation.

4.7 Group-Aware Validation Comparison

Repeat the idea for entities using `GroupKFold` or `StratifiedGroupKFold`.

```
from sklearn.model_selection import GroupKFold

group_cv = cross_val_score(model, X, y, groups=patient_id,
    cv=GroupKFold(5), scoring='roc_auc').mean()
```

If scores fall sharply when entities cannot cross folds, the model was memorizing entities.

4.8 Identifier Overlap Checks

Simple set arithmetic catches a surprising amount of leakage.

```
for key in ['user_id', 'patient_id', 'device_id', 'merchant_id']:
    overlap = set(train[key]) & set(test[key])
    print(key, 'overlap:', len(overlap), 'of', test[key].nunique())
```

Also test whether ID-like fields themselves are predictive. If row index, customer ID or filename predicts the label, the data ordering or collection process encodes it.

4.9 Duplicate and Near-Duplicate Detection

Exact duplicates: hash rows or files.

```
train_hash = set(pd.util.hash_pandas_object(train.drop(columns=['target']), index=False))
test_hash = pd.util.hash_pandas_object(test.drop(columns=['target']), index=False)
print('Exact duplicate rate in test:', test_hash.isin(train_hash).mean())
```

Near duplicates:

- Tabular: nearest-neighbor distance between test and train rows. Unusually small distances signal copies.
- Text: MinHash with locality-sensitive hashing, or embedding cosine similarity above a threshold.
- Images: perceptual hashing (pHash, dHash) or embedding similarity.
- Audio and video: fingerprinting and frame-level hashing.

```
from sklearn.neighbors import NearestNeighbors

nn = NearestNeighbors(n_neighbors=1).fit(X_train_scaled)
dist, _ = nn.kneighbors(X_test_scaled)
print('Share of test rows with near-zero distance:', (dist < 1e-6).mean())
```

4.10 Pipeline Audit for Preprocessing Leakage

The fix is structural, and so is the detection: look for any `.fit()` or `.fit_transform()` call that touches data before the split.

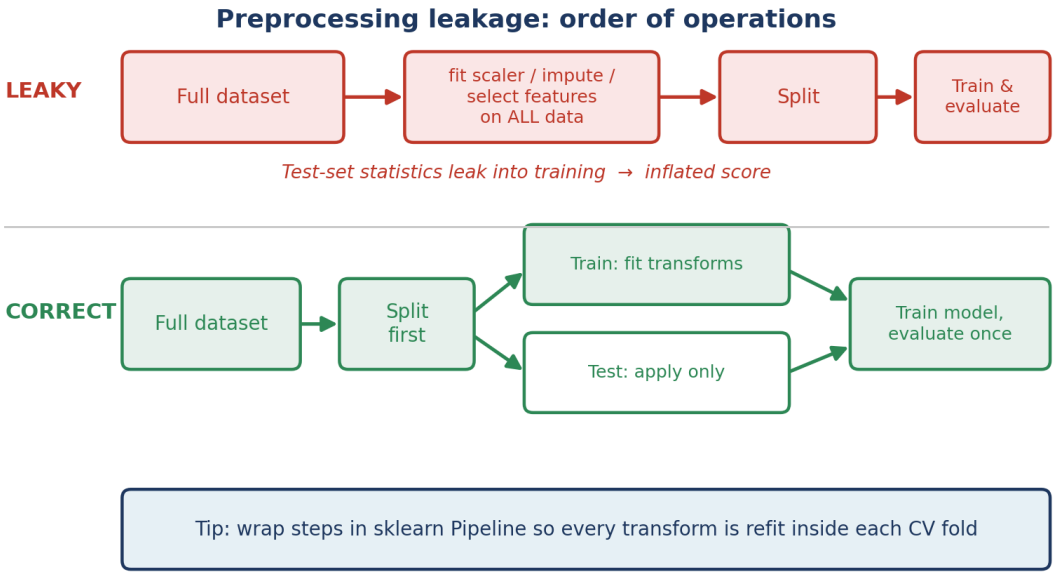


Figure 6. Leaky versus correct order of operations.

Wrong

```
X_scaled = StandardScaler().fit_transform(X)       # fit on everything
X_tr, X_te, y_tr, y_te = train_test_split(X_scaled, y)
```

Right

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scale', StandardScaler()),
    ('clf', LogisticRegression(max_iter=1000)),
])
# transforms are refit inside each fold
scores = cross_val_score(pipe, X, y, cv=5, scoring='roc_auc')
```

A practical trick: **run the pipeline twice**, once with the leaky ordering and once with the correct one. The score gap tells you how much leakage you had. For imbalanced data, apply SMOTE only inside the training fold with `imblearn.pipeline.Pipeline`, never before splitting.

4.11 Target Encoding Audit

Target encoding is a leakage magnet. If you encode a category by the mean of the target, rows contribute to their own encoding. Detect it by checking whether a categorical feature's encoded version is far more predictive than the raw category on a clean holdout. Prevent it with out-of-fold encoding, smoothing and noise.

4.12 Point-in-Time Correctness Checks

For features built from event logs or warehouses:

- Each feature value must use only data with a timestamp earlier than the prediction timestamp (the **as-of time**).
- Verify with an assertion: for sampled rows, recompute the feature from raw data cut off at the as-of time and compare to the stored value.
- Check that join keys include time conditions (as-of joins), not just entity IDs.

```
merged = pd.merge_asof(
    events.sort_values('event_time'),
    features.sort_values('feature_time'),
    left_on='event_time', right_on='feature_time',
    by='user_id', direction='backward'
)
assert (merged['feature_time'] <= merged['event_time']).all()
```

4.13 Artifact and Shortcut Detection

In unstructured data, models love shortcuts.

- **Metadata-only baseline:** train on file size, dimensions, timestamps, source and filename, and see how far it goes.
- **Image models:** inspect saliency maps (Grad-CAM, integrated gradients). Attention on corners, text overlays or borders instead of the region of interest signals artifact leakage.
- **Text models:** check top n-grams or tokens by importance for boilerplate, template headers, signatures or dataset-specific phrases.
- **Occlusion test:** mask the suspected artifact region and watch whether performance collapses.

4.14 Ablation and Leave-One-Source-Out Tests

- Drop the suspect feature or feature group and measure performance. A huge fall means heavy dependence and demands justification.
- If data comes from multiple sources (hospitals, sensors, regions), train on some sources and test on a held-out source. Large drops mean the model learned source identity.

4.15 Learning-Curve and Complexity Checks

Try progressively simpler models. If a depth-1 tree or a logistic regression on a few features matches a deep ensemble, a single leaky feature is probably doing all the work. Likewise, if performance is already near ceiling with 1% of the training data, suspect leakage or duplicates.

4.16 Online vs Offline Reconciliation

The ultimate test is reality.

- Run the model in **shadow mode** and compare live metrics to offline estimates.
- Log serving features and compare their distribution to training features (PSI, KS tests or Jensen-Shannon distance).
- Recompute features offline for production requests and diff them against what was actually served.

5. Detecting Contamination in LLMs and Foundation Models

Standard tabular techniques are not enough here. Common approaches:

- **N-gram overlap:** search for long exact n-gram matches (commonly 8 to 13 tokens) between benchmark items and the training corpus.
- **Embedding similarity:** find paraphrased or translated versions of benchmark questions in training data.
- **Perplexity and likelihood tests:** unusually low perplexity on benchmark items compared with fresh, similar items suggests memorization.
- **Completion probing:** give the model the first part of a test item and see whether it reproduces the rest verbatim, or guesses the answer without the question.
- **Membership inference attacks:** statistical tests estimating whether a specific sample was in the training set.
- **Rewritten and fresh benchmarks:** evaluate on perturbed variants (changed numbers, reordered options) or questions written after the training cutoff. A large drop from the original benchmark indicates contamination.
- **Dynamic and held-out evaluation:** private test sets and time-stamped benchmarks built from events after the cutoff.

No single detector is conclusive. Combine them, and report contamination analysis alongside benchmark numbers.

06. A Systematic Leakage Audit Workflow

Use this sequence on every serious project:

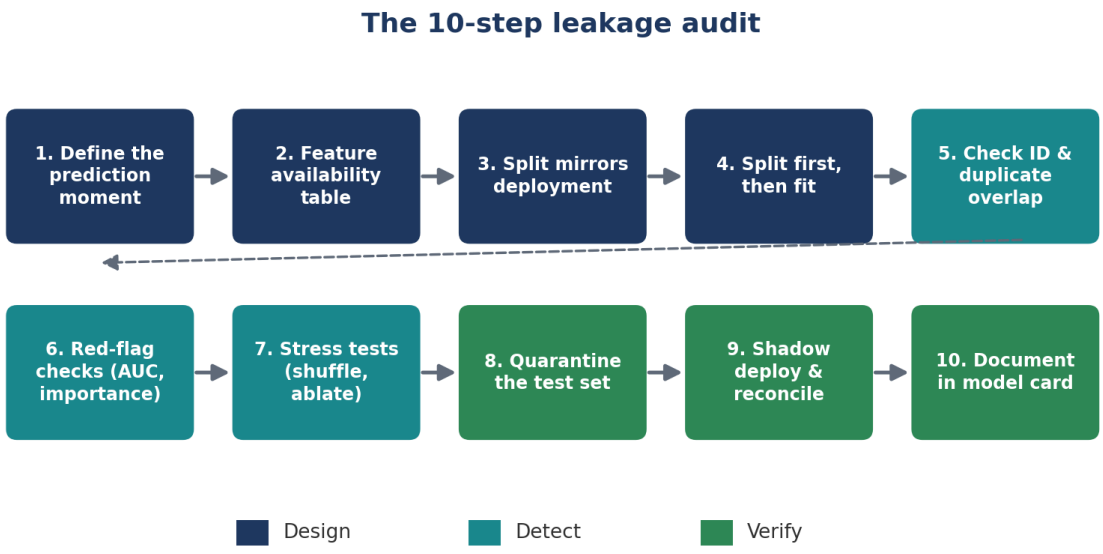


Figure 7. The 10-step leakage audit.

1. **Define the prediction moment.** Write down exactly when and where the model will be called, and what data exists at that instant.
2. **Build a feature availability table.** For each feature: source, timestamp, latency, and whether it exists at prediction time.
3. **Choose the split to mirror deployment.** Time-based if predicting the future, group-based for new entities, source-based for new sites.
4. **Split first, then touch data.** All fitting happens inside training folds only.
5. **Check overlap.** IDs, exact duplicates, near duplicates across splits.
6. **Run the red-flag checks.** Single-feature AUC, importance audit, adversarial validation.
7. **Run the stress tests.** Shuffled labels, alternative splits, leave-one-source-out, ablation.
8. **Quarantine the test set.** Touch it once, at the end, for the final estimate.
9. **Verify in production.** Shadow deployment and offline-online reconciliation.
10. **Document everything** in a model card or evaluation report, including what you checked and what you could not check.

07. Prevention: Designing Leakage Out of the System

Detection is a safety net. Prevention is better engineering.

Process

- Hold out a final test set and lock it away. Treat it like a production secret.
- Use nested cross-validation when tuning and estimating performance on limited data.
- Add independent review: someone who did not build the pipeline tries to break it.
- Keep an experiment log so repeated peeking at the test set becomes visible.

Code

- Use `Pipeline` and `ColumnTransformer` so fitted transforms are scoped to training folds.
- Write unit tests: no ID overlap, `max(train_time) < min(test_time)`, no duplicates across splits.
- Make split logic a single, versioned, reviewed function instead of ad hoc notebook cells.

```
def assert_no_leakage(train, test, id_col, time_col):
    assert set(train[id_col]).isdisjoint(test[id_col]), 'Entity overlap between splits'
    assert train[time_col].max() < test[time_col].min(), 'Temporal overlap between splits'
```

Data and infrastructure

- Adopt a feature store with point-in-time joins.
- Version datasets and snapshots so features can be reproduced as they were.
- Use the same feature code for training and serving to eliminate skew.
- Add data contracts that document what each field means and when it is populated.

Culture

- Reward skepticism. Celebrate the engineer who finds the leak, not the one who posts the highest score.
- Ask in every review: *Why is this number this good?*

8. Tools Worth Knowing

Purpose	Tools
Pipelines and CV	scikit-learn `Pipeline`, `GroupKFold`, `TimeSeriesSplit`, imbalanced-learn pipelines
Interpretability	SHAP, LIME, Captum, scikit-learn permutation importance
Data validation	Great Expectations, Pandera, TensorFlow Data Validation, Deepchecks
Drift and skew	Evidently, WhyLabs, NannyML, Alibi Detect
Feature stores	Feast, Tecton, Databricks Feature Store, Vertex AI Feature Store
Near-duplicate detection	datasketch (MinHash), imagehash, FAISS and embedding search
Experiment tracking	MLflow, Weights and Biases, DVC

Deepchecks ships built-in checks for train-test overlap, feature-label correlation and date leakage, which makes it a good first automated sweep.

9. Case Studies

The ICU model that learned the lab order. A model predicting sepsis scored brilliantly using the presence of certain lab tests. Doctors ordered those tests *because* they already suspected sepsis. The feature encoded clinician behavior, a downstream consequence of the label, and vanished as a signal in a different workflow.

The pneumonia classifier that learned the hospital. Chest X-ray models have shown strong performance partly by recognizing which hospital produced the image, since disease prevalence differed between sites. Leave-one-hospital-out evaluation exposed the drop.

The churn model with tomorrow's data. A subscription company built features from a table overwritten nightly. Training used the latest snapshot, so features such as plan status already reflected cancellation. A point-in-time rebuild cut apparent AUC from 0.97 to 0.78, the honest number.

The leaderboard that was not. In competitions, row order, ID sequences and file timestamps have repeatedly turned out to predict labels. Technically legal, practically useless, and a reminder that the data-generating process can leak through anything.

> **Note on case studies**
> These are composite, illustrative scenarios based on well-documented patterns of leakage, not reports of specific named incidents. Figures are examples.

10. Practitioner Checklist

Before modeling

- ☐ Prediction moment and available data documented
- ☐ Feature availability table completed and reviewed
- ☐ Split strategy matches deployment (time, group, source)
- ☐ Test set locked

During modeling

- ☐ All preprocessing inside pipelines
- ☐ No fitting before splitting (scaling, imputing, selection, encoding, resampling)
- ☐ ID, exact duplicate and near-duplicate overlap checked
- ☐ Single-feature AUC and importance audit reviewed
- ☐ Adversarial validation run
- ☐ Shuffled-label test passes
- ☐ Random vs time vs group CV compared
- ☐ Ablation of top features done

Before release

- ☐ Test set evaluated once
- ☐ Metrics plausible against baselines and domain knowledge
- ☐ Shadow deployment planned
- ☐ Train-serving skew monitoring in place
- ☐ Leakage checks documented in the model card

11. Common Misconceptions

- *"Cross-validation protects me from leakage."* No. CV only protects against overfitting to one split. If leakage is in the features or preprocessing, every fold inherits it.
- *"Leakage always produces absurd accuracy."* No. Subtle leakage can add a few points and still pass every smell test.
- *"Removing the obvious column fixes it."* Not necessarily. Other features can be proxies for the removed one.
- *"A random held-out set is independent."* Not when rows are correlated through users, time, devices or sources.
- *"This only matters for tabular data."* It matters at least as much for images, text, audio and LLM evaluation.

Conclusion

Data leakage is not an exotic corner case. It is the default failure mode of ML evaluation, and it thrives wherever teams move fast and trust a good-looking metric. The remedy combines **discipline** (define the prediction moment, split first, guard the test set), **diagnostics** (single-feature tests, adversarial validation, shuffled labels, alternative splits, overlap checks), **infrastructure** (pipelines, point-in-time features, data contracts) and **skepticism**.

The best model is not the one with the highest offline score. It is the one whose offline score you can trust to survive contact with reality.

Key Takeaways

1. Leakage means the model had access to information it will not have in production, or the test set was not truly independent.
2. Learn the taxonomy: target, preprocessing, temporal, group, duplicate, label, evaluation and contamination leakage.
3. Treat unusually good results as a bug until proven otherwise.
4. Combine detectors: single-feature AUC, permutation importance, adversarial validation, shuffled labels and split comparisons.
5. Split first, fit later, and keep every transformation inside a pipeline.
6. Verify with shadow deployment and continuous train-serving skew monitoring.

Further reading: Kaufman et al., "Leakage in Data Mining: Formulation, Detection, and Avoidance"; Kapoor and Narayanan, "Leakage and the Reproducibility Crisis in ML-based Science"; Kaggle's data leakage micro-course; scikit-learn documentation on common pitfalls.